# Exercise 6

Advanced Methods for Regression and Classification

Kyzer Gerez

19.11.2025

```
library(fastDummies)

# read csv and drop duration col
d <- subset(read.csv2("bank.csv"), select=-c(duration))

# check for na
rownames(d[rowSums(is.na(d)) > 0,])
```

```
## character(0)
```

```
# convert month to numeric representation
d["month"] <- match(d[["month"]], tolower(month.abb))

# convert binary variables to numeric representation
d$default <- ifelse(d$default=="no", 0, 1)
d$housing <- ifelse(d$housing=="no", 0, 1)
d$loan <- ifelse(d$loan=="no", 0, 1)
d$y <- ifelse(d$y=="no", 0, 1)

# apply one-hot encoding to rest of categorical predictors
d.encoding <- dummy_cols(d,
                         remove_selected_columns=TRUE)

# split into 3/4 train, 1/4 test
smp_size <- floor(3/4 * nrow(d))

# set the seed to make partition reproducible
set.seed(1)
train_ind <- sample(seq_len(nrow(d.encoding)), size = smp_size)

train <- d.encoding[train_ind, ]
test <- d.encoding[-train_ind, ]
```

1. *Linear Discrimant Analysis (LDA)*: function `lda` from `library(MASS)`

   (a) Apply `lda()` to the training set. Is any data preprocessing necessary or advisable?

   Yes, data preprocessing is necessary since LDA requires that all predictors be in numeric form and scaled. One hot-encoding should be used for nominal predictors such as job and marital, while the binary predictors and ordinal month predictor can simply be converted to a numerical form. Education could also be treated as an ordinal variable, but it's unclear how unknown values should be handled; therefore, it's treated as a nominal variable instead.

Additionally, unknown values will be treated as their own category and will not be imputed.

```
library(MASS)

mod.lda <- lda(x = train[,-11], grouping = train$y) # remove y for x param
```

```
## Warning in lda.default(x, grouping, ...): variables are collinear
```

(b) Compute the evaluation measures for the training data. What do you conclude?

```
lda.train.pred <- predict(mod.lda, newdata=train[,-11])

# create confusion matrix
library(caret)

lda.train.confusion <- confusionMatrix(data=lda.train.pred$class,
                                       reference=as.factor(train$y),
                                       positive = "1")
lda.train.confusion
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 2968  327
##          1   35   60
##
##                Accuracy : 0.8932
##                  95% CI : (0.8823, 0.9034)
##     No Information Rate : 0.8858
##     P-Value [Acc > NIR] : 0.09198
##
##                   Kappa : 0.2136
##
##  Mcnemar's Test P-Value : < 2e-16
##
##             Sensitivity : 0.15504
##             Specificity : 0.98834
##          Pos Pred Value : 0.63158
##          Neg Pred Value : 0.90076
##              Prevalence : 0.11416
##          Detection Rate : 0.01770
##    Detection Prevalence : 0.02802
##       Balanced Accuracy : 0.57169
##
##        'Positive' Class : 1
##
```

The model tends to predict no due to the unbalanced dataset. This can be seen from the high accuracy compared to the lower values for sensitivity and balanced accuracy.

(c) Predict the group membership for the test data and compute the evaluation measures. What do you conclude?

```
lda.test.pred <- predict(mod.lda, newdata=test[,-11])

lda.test.confusion <- confusionMatrix(data=lda.test.pred$class,
```

```
                                        reference=as.factor(test$y),
                                        positive = "1")
lda.test.confusion
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 986 110
##          1  11  24
##
##                Accuracy : 0.893
##                  95% CI : (0.8735, 0.9104)
##     No Information Rate : 0.8815
##     P-Value [Acc > NIR] : 0.1242
##
##                   Kappa : 0.2471
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.17910
##             Specificity : 0.98897
##          Pos Pred Value : 0.68571
##          Neg Pred Value : 0.89964
##              Prevalence : 0.11848
##          Detection Rate : 0.02122
##    Detection Prevalence : 0.03095
##       Balanced Accuracy : 0.58404
##
##        'Positive' Class : 1
##
```

Again, the same results can be seen in that the model overpredicts for no due to the unbalanced dataset.

2. Assume that we want to maximize the balanced accuracy. This could be achieved by selecting a balanced training set, i.e., the same number of observations from both groups, to train the classifier. Thus, repeat tasks 1.(a)-(c) for:

```
# setup #
n_0 <- length(which(d.encoding$y==0)) # max
n_1 <- length(which(d.encoding$y==1)) # min

train.only.0 <- d.encoding[which(d.encoding$y==0),]
train.only.1 <- d.encoding[which(d.encoding$y==1),]

# get undersampled train set
under.indices <- sample(seq_len(nrow(train.only.0)), n_1)
under.0 <- train.only.0[under.indices,]
train.under <- rbind(train.only.1,
                     under.0)

# get oversampled train set
over.indices <- sample(seq_len(nrow(train.only.1)), n_0, replace=TRUE)
over.1 <- train.only.1[over.indices,]
train.over <- rbind(train.only.0,
```

```
                      over.1)
```

(a) "Undersampling": Suppose the number of samples in the original training set is $n_1$ and $n_2$ for the two groups. Now select from this training set $\min(n_1, n_2)$ samples from each group. The test set is unchanged.

```
# train model
mod.lda.under <- lda(x = train.under[,-11], grouping = train.under$y)
```

```
## Warning in lda.default(x, grouping, ...): variables are collinear
```

```
# predict on train and eval
lda.under.train.pred <- predict(mod.lda.under, newdata=train[,-11])
lda.under.train.confusion <- confusionMatrix(data=lda.under.train.pred$class,
                                             reference=as.factor(train$y),
                                             positive = "1")
lda.under.train.confusion
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 2052  140
##          1  951  247
##
##                Accuracy : 0.6782
##                  95% CI : (0.6621, 0.6939)
##     No Information Rate : 0.8858
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : 0.1681
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.63824
##             Specificity : 0.68332
##          Pos Pred Value : 0.20618
##          Neg Pred Value : 0.93613
##              Prevalence : 0.11416
##          Detection Rate : 0.07286
##    Detection Prevalence : 0.35339
##       Balanced Accuracy : 0.66078
##
##        'Positive' Class : 1
##
```

```
# predict on test and eval
lda.under.test.pred <- predict(mod.lda.under, newdata=test[,-11])
lda.under.test.confusion <- confusionMatrix(data=lda.under.test.pred$class,
                                            reference=as.factor(test$y),
                                            positive = "1")
lda.under.test.confusion
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
```

```
##          0 690  55
##          1 307  79
##
##                Accuracy : 0.6799
##                  95% CI : (0.6519, 0.7071)
##     No Information Rate : 0.8815
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : 0.1553
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.58955
##             Specificity : 0.69208
##          Pos Pred Value : 0.20466
##          Neg Pred Value : 0.92617
##              Prevalence : 0.11848
##          Detection Rate : 0.06985
##    Detection Prevalence : 0.34129
##       Balanced Accuracy : 0.64081
##
##        'Positive' Class : 1
##
```

(b) "Oversampling": Select from the training set $\max(n_1, n_2)$ samples from each group. From the smaller group you need to sample with replacement. The test set is unchanged.

```
# train model
mod.lda.over <- lda(x = train.over[,-11], grouping = train.over$y)
```

```
## Warning in lda.default(x, grouping, ...): variables are collinear
```

```
# predict on train and eval
lda.over.train.pred <- predict(mod.lda.over, newdata=train[,-11])
lda.over.train.confusion <- confusionMatrix(data=lda.over.train.pred$class,
                                            reference=as.factor(train$y),
                                            positive = "1")
lda.over.train.confusion
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 2055  146
##          1  948  241
##
##                Accuracy : 0.6773
##                  95% CI : (0.6613, 0.693)
##     No Information Rate : 0.8858
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : 0.1614
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.62274
```

```
##             Specificity : 0.68432
##          Pos Pred Value : 0.20269
##          Neg Pred Value : 0.93367
##              Prevalence : 0.11416
##          Detection Rate : 0.07109
##    Detection Prevalence : 0.35074
##       Balanced Accuracy : 0.65353
##
##        'Positive' Class : 1
##
```

```
# predict on test and eval
lda.over.test.pred <- predict(mod.lda.over, newdata=test[,-11])
lda.over.test.confusion <- confusionMatrix(data=lda.over.test.pred$class,
                                           reference=as.factor(test$y),
                                           positive = "1")
lda.over.test.confusion
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 692  55
##          1 305  79
##
##                Accuracy : 0.6817
##                  95% CI : (0.6537, 0.7088)
##     No Information Rate : 0.8815
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : 0.1569
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.58955
##             Specificity : 0.69408
##          Pos Pred Value : 0.20573
##          Neg Pred Value : 0.92637
##              Prevalence : 0.11848
##          Detection Rate : 0.06985
##    Detection Prevalence : 0.33952
##       Balanced Accuracy : 0.64182
##
##        'Positive' Class : 1
##
```

Which strategy is more successful, and why?

The oversampling strategy performed slightly better for the test set, while undersampling resulted in a higher balanced accuracy for the train set. This is a bit unexpected since oversampling should lead to more overfitting. But it being better for the test set makes sense. The disadvantage of undersampling is that information from the majority class is discarded, which can lead to weaker predictive power.

3. *Quadratic Discrimant Analysis (QDA)*: function `qda` from `library(MASS)`. Use oversampling and undersampling for `qda()` and report the evaluation measures for the test set. What do you conclude?

```r
library(MASS)

### QDA-specific preprocessing ###
# check if class 0 predictors are full rank
X0 <- as.matrix(train[train$y == 0, -which(names(train) == "y")])
k <- qr(X0)$rank
p <- ncol(X0)
k; p
```

```
## [1] 32
```

```
## [1] 36
```

```r
print("k<p, so class 0 predictors are not full rank.")
```

```
## [1] "k<p, so class 0 predictors are not full rank."
```

```r
# check for linear combos
remove.cols <- findLinearCombos(X0)$remove

# remove predictors 25 29 32 36
train.qda <- train[, -remove.cols]
train.qda.under <- train.under[, -remove.cols]
train.qda.over <- train.over[, -remove.cols]
test.qda <- test[, -remove.cols]

# find columns with near-zero variance
nzv_cols <- nearZeroVar(X0, saveMetrics = TRUE)

# remove columns with near-zero variance
train.qda <- train.qda[, !names(train.qda) %in%
                         rownames(nzv_cols[nzv_cols$nzv == TRUE, ])]
train.qda.under <- train.qda.under[, !names(train.qda.under) %in%
                         rownames(nzv_cols[nzv_cols$nzv == TRUE, ])]
train.qda.over <- train.qda.over[, !names(train.qda.over) %in%
                         rownames(nzv_cols[nzv_cols$nzv == TRUE, ])]
test.qda <- test.qda[, !names(test.qda) %in%
                         rownames(nzv_cols[nzv_cols$nzv == TRUE, ])]

### undersampling ###
# train model
mod.qda.under <- qda(y ~ ., data=train.qda.under)

# predict on train and eval
qda.under.train.pred <- predict(mod.qda.under, newdata=train.qda[,-9])
qda.under.train.confusion <- confusionMatrix(data=qda.under.train.pred$class,
                                             reference=as.factor(train.qda$y),
                                             positive = "1")
qda.under.train.confusion
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 2104  144
##          1  899  243
```

```
##
##                Accuracy : 0.6923
##                  95% CI : (0.6765, 0.7078)
##     No Information Rate : 0.8858
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : 0.1776
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.62791
##             Specificity : 0.70063
##          Pos Pred Value : 0.21278
##          Neg Pred Value : 0.93594
##              Prevalence : 0.11416
##          Detection Rate : 0.07168
##    Detection Prevalence : 0.33687
##       Balanced Accuracy : 0.66427
##
##        'Positive' Class : 1
##
```

```
# predict on test and eval
qda.under.test.pred <- predict(mod.qda.under, newdata=test.qda[,-9])
qda.under.test.confusion <- confusionMatrix(data=qda.under.test.pred$class,
                                            reference=as.factor(test.qda$y),
                                            positive = "1")
qda.under.test.confusion
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 703  61
##          1 294  73
##
##                Accuracy : 0.6861
##                  95% CI : (0.6582, 0.7131)
##     No Information Rate : 0.8815
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : 0.1426
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.54478
##             Specificity : 0.70512
##          Pos Pred Value : 0.19891
##          Neg Pred Value : 0.92016
##              Prevalence : 0.11848
##          Detection Rate : 0.06454
##    Detection Prevalence : 0.32449
##       Balanced Accuracy : 0.62495
##
##        'Positive' Class : 1
```

```
##
## oversampling ##
# train model
mod.qda.over <- qda(y ~ ., data=train.qda.over)

# predict on train and eval
qda.over.train.pred <- predict(mod.qda.over, newdata=train.qda[,-9])
qda.over.train.confusion <- confusionMatrix(data=qda.over.train.pred$class,
                                            reference=as.factor(train.qda$y),
                                            positive = "1")
qda.over.train.confusion
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 1991  118
##          1 1012  269
##
##                Accuracy : 0.6667
##                  95% CI : (0.6505, 0.6825)
##     No Information Rate : 0.8858
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : 0.1785
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.69509
##             Specificity : 0.66300
##          Pos Pred Value : 0.20999
##          Neg Pred Value : 0.94405
##              Prevalence : 0.11416
##          Detection Rate : 0.07935
##    Detection Prevalence : 0.37788
##       Balanced Accuracy : 0.67905
##
##        'Positive' Class : 1
##
```

```
# predict on test and eval
qda.over.test.pred <- predict(mod.qda.over, newdata=test.qda[,-9])
qda.over.test.confusion <- confusionMatrix(data=qda.over.test.pred$class,
                                           reference=as.factor(test.qda$y),
                                           positive = "1")
qda.over.test.confusion
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 665  47
##          1 332  87
##
```

```
##                Accuracy : 0.6649
##                  95% CI : (0.6365, 0.6924)
##     No Information Rate : 0.8815
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : 0.1647
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.64925
##             Specificity : 0.66700
##          Pos Pred Value : 0.20764
##          Neg Pred Value : 0.93399
##              Prevalence : 0.11848
##          Detection Rate : 0.07692
##    Detection Prevalence : 0.37047
##       Balanced Accuracy : 0.65813
##
##        'Positive' Class : 1
##
```

Significant additional data-preprocessing was needed for QDA to work. QDA is brittle in that it can't handle predictors that have near zero variance or are linear combinations of other predictors.

The oversampling strategy outperformed for both the train and test sets.

4. *Regularized Discrimant Analysis (RDA)*: function `rda` from `library(klaR)` Use oversampling and undersampling for `rda()` and report the evaluation measures for the test set. What do you conclude? Interpret the meaning of the resulting tuning parameters `gamma` and `lambda`.

```
library(klaR)

## undersampling ##
# train model
mod.rda.under <- rda(x = train.under[,-11], grouping = train.under$y)

# predict on train and eval
rda.under.train.pred <- predict(mod.rda.under, newdata=train[,-11])
rda.under.train.confusion <- confusionMatrix(data=rda.under.train.pred$class,
                                             reference=as.factor(train$y),
                                             positive = "1")
rda.under.train.confusion
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0  165   14
##          1 2838  373
##
##                Accuracy : 0.1587
##                  95% CI : (0.1466, 0.1714)
##     No Information Rate : 0.8858
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : 0.0045
```

```
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.96382
##             Specificity : 0.05495
##          Pos Pred Value : 0.11616
##          Neg Pred Value : 0.92179
##              Prevalence : 0.11416
##          Detection Rate : 0.11003
##    Detection Prevalence : 0.94720
##       Balanced Accuracy : 0.50938
##
##        'Positive' Class : 1
##
```

```
# predict on test and eval
rda.under.test.pred <- predict(mod.rda.under, newdata=test[,-11])
rda.under.test.confusion <- confusionMatrix(data=rda.under.test.pred$class,
                                            reference=as.factor(test$y),
                                            positive = "1")
rda.under.test.confusion
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0  57   9
##          1 940 125
##
##                Accuracy : 0.1609
##                  95% CI : (0.14, 0.1837)
##     No Information Rate : 0.8815
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : -0.0025
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.93284
##             Specificity : 0.05717
##          Pos Pred Value : 0.11737
##          Neg Pred Value : 0.86364
##              Prevalence : 0.11848
##          Detection Rate : 0.11052
##    Detection Prevalence : 0.94164
##       Balanced Accuracy : 0.49500
##
##        'Positive' Class : 1
##
```

```
## oversampling ##
# train model
mod.rda.over <- rda(x = train.over[,-11], grouping = train.over$y)

# predict on train and eval
```

```
rda.over.train.pred <- predict(mod.rda.over, newdata=train[,-11])
rda.over.train.confusion <- confusionMatrix(data=rda.over.train.pred$class,
                                            reference=as.factor(train$y),
                                            positive = "1")
rda.over.train.confusion
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 1708  180
##          1 1295  207
##
##                Accuracy : 0.5649
##                  95% CI : (0.548, 0.5817)
##     No Information Rate : 0.8858
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : 0.046
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.53488
##             Specificity : 0.56876
##          Pos Pred Value : 0.13782
##          Neg Pred Value : 0.90466
##              Prevalence : 0.11416
##          Detection Rate : 0.06106
##    Detection Prevalence : 0.44307
##       Balanced Accuracy : 0.55182
##
##        'Positive' Class : 1
##
```

```
# predict on test and eval
rda.over.test.pred <- predict(mod.rda.over, newdata=test[,-11])
rda.over.test.confusion <- confusionMatrix(data=rda.over.test.pred$class,
                                            reference=as.factor(test$y),
                                            positive = "1")
rda.over.test.confusion
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 574  51
##          1 423  83
##
##                Accuracy : 0.5809
##                  95% CI : (0.5515, 0.6099)
##     No Information Rate : 0.8815
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : 0.0886
##
```

```
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.61940
##             Specificity : 0.57573
##          Pos Pred Value : 0.16403
##          Neg Pred Value : 0.91840
##              Prevalence : 0.11848
##          Detection Rate : 0.07339
##    Detection Prevalence : 0.44739
##       Balanced Accuracy : 0.59757
##
##        'Positive' Class : 1
##
```

```
# output tuning parameters
rda.tuning.params <- data.frame(
  model = c("under", "over"),
  gamma = c(mod.rda.under$regularization[["gamma"]],
            mod.rda.over$regularization[["gamma"]]),
  lambda = c(mod.rda.under$regularization[["lambda"]],
             mod.rda.over$regularization[["lambda"]])
)
rda.tuning.params
```

```
##   model     gamma    lambda
## 1 under 0.9590229 0.7392812
## 2  over 0.9998678 0.8942118
```

The oversampling strategy performed significantly better than undersampling for both the train and test sets. RDA is a compromise between QDA and LDA. The lambda values for the two sampling strategies indicate that the estimated error rate is minimized using values that shape the models to being closer to LDA than QDA. The high values for gamma for both sampling strategies show that the covariance structure was shrunk to a more spherical shape. This means that predictors are more equally weighted compared to pure QDA and LDA.

```
# summary of results
summary <- data.frame(
  model = c("lda", "lda.under", "lda.over", "qda.under", "qda.over",
            "rda.under", "rda.over"),
  train_balanced_acc = c(lda.train.confusion$byClass[["Balanced Accuracy"]],
                         lda.under.train.confusion$byClass[["Balanced Accuracy"]],
                         lda.over.train.confusion$byClass[["Balanced Accuracy"]],
                         qda.under.train.confusion$byClass[["Balanced Accuracy"]],
                         qda.over.train.confusion$byClass[["Balanced Accuracy"]],
                         rda.under.train.confusion$byClass[["Balanced Accuracy"]],
                         rda.over.train.confusion$byClass[["Balanced Accuracy"]]),
  test_balanced_acc = c(lda.test.confusion$byClass[["Balanced Accuracy"]],
                        lda.under.test.confusion$byClass[["Balanced Accuracy"]],
                        lda.over.test.confusion$byClass[["Balanced Accuracy"]],
                        qda.under.test.confusion$byClass[["Balanced Accuracy"]],
                        qda.over.test.confusion$byClass[["Balanced Accuracy"]],
                        rda.under.test.confusion$byClass[["Balanced Accuracy"]],
                        rda.over.test.confusion$byClass[["Balanced Accuracy"]])
  )
summary
```

```
##       model train_balanced_acc test_balanced_acc
## 1       lda          0.5716919         0.5840357
## 2 lda.under          0.6607798         0.6408142
## 3  lda.over          0.6535274         0.6418172
## 4 qda.under          0.6642698         0.6249457
## 5  qda.over          0.6790471         0.6581274
## 6 rda.under          0.5093847         0.4950037
## 7  rda.over          0.5518241         0.5975651
```

Overall, QDA using the oversampling strategy outperformed all other models for both the train and test sets. Though comparisons between QDA and the other methods aren't on a completely sound basis since QDA required train and test sets with less predictors than the sets used to create the LDA and RDA models.